

Introduction à l'algorithmique et à la programmation

Les alternatives en C

Département informatique - IUT de Lannion

Rappel : tests et itérations

- Les tests et itérations permettent de contrôler la suite séquentielle des instructions
- tests : *si un booléen est satisfait faire ... sinon ...*
- itérations (répétition) : *tant qu'un booléen est satisfait faire ...*

booléen est soit :

- une variable (ou une expression) de type booléen
- une condition

Tout repose sur les expressions booléennes et les opérateurs associés.

Plan

- 1 Les expressions booléennes en C
- 2 Les alternatives en C

Les expressions booléennes en C

- Rappel : le type booléen n'existe pas en C
- Une comparaison renvoie un booléen et s'écrit :
`<exp1> <opérateur> <exp2>`
- Les expressions doivent être de même type
- Les opérateurs de comparaison sont :
 - égalité : `<exp1> == <exp2>`
 - inégalité : `<, >, <=, >=`
 - différence : `<exp1> != <exp2>`
- Les opérateurs logiques (sur les booléens) sont :
 - négation logique : `! <bool1>`
 - conjonction : `<bool1> && <bool2>`
 - disjonction : `<bool1> || <bool2>`
 - disjonction exclusive : `<bool1> ^ <bool2>`
- La librairie `stdbool.h` n'est pas nécessaire pour les comparaisons, uniquement pour les variables booléennes

Les expressions booléennes en C

- Cas particulier des chaînes de caractères
- Si `ch1` et `ch2` sont deux chaînes de caractères (constantes ou variables)
- Pour comparer deux chaînes de caractères on utilise l'instruction `strcmp(ch1, ch2)` de la librairie `string.h` (ajouter `#include <string.h>`)
- La comparaison selon l'ordre lexicographique du dictionnaire
- L'instruction retourne :
 - entier négatif si `ch1 < ch2`
 - 0 si `ch1 == ch2`
 - entier strictement positif si `ch1 > ch2`

Plan

- ① Les expressions booléennes en C
- ② Les alternatives en C

Les alternatives en C

- Si ... alors :

```
if (<expression booléenne>
{
    <liste d'instructions 1>
}
```

- S'il n'y a qu'une instruction ça peut s'écrire :

```
if (<expression booléenne>
    <une instruction>;
```

éviter... difficulté de maintenanc

- Attention à l'erreur classique :

```
if (<expression booléenne>);
    <une instruction>;
```

instruction vide

L'instruction sera faite de toute façon.

Les alternatives en C

- Si ... alors ... sinon :

```
<instruction 0>
```

```
if (<expression booléenne>)
```

```
{
```

```
    <liste d'instructions 1>
```

```
}
```

```
else
```

```
{
```

```
    <liste d'instructions 2>
```

```
}
```

```
<instruction n>
```


Parenthèse sur les accolades

C'est une question de norme et de goût.

```
int main()  
{  
    <instructions>  
}
```

```
int main(){  
    <instructions>  
}
```

Parenthèse sur les accolades

C'est une question de norme et de goût.

```
if (<expression booléenne>)  
{  
    <liste d'instructions 1>  
}  
else  
{  
    <liste d'instructions 2>  
}
```

```
if (<expression booléenne>) {  
    <liste d'instructions 1>  
} else {  
    <liste d'instructions 2>  
}
```

Les alternatives imbriquées

- Une alternative est une instruction, elle peut donc être imbriquée dans une autre structure alternative
- Par exemple

```
if <condition1>
{
    <liste1 d'instructions>
    if <condition2>
    {
        <liste2 d'instructions>
    }
}
else
{
    <liste3 d'instructions>
}
```

Les alternatives imbriquées

- Une alternative est une instruction, elle peut donc être imbriquée dans une autre structure alternative
- Par exemple

```
if <condition1>
{
    <liste1 d'instructions>
    if <condition2>
    {
        <liste2 d'instructions>
    }
    else
    {
        <liste3 d'instructions>
    }
}
```

Les alternatives imbriquées

- Une alternative est une instruction, elle peut donc être imbriquée dans une autre structure alternative
- Par exemple

```
if <condition1>
{
    <liste1 d'instructions>
}
else if <condition2>
{
    <liste2 d'instructions>
}
else
{
    <liste3 d'instructions>
}
```

Les alternatives imbriquées

```
if (<expression booléenne 1>)  
{  
    <liste d'instructions 1>  
}  
else if (<expression booléenne 2>)  
{  
    <liste d'instructions 2>  
}  
else if (<expression booléenne 3>)  
{  
    <liste d'instructions 3>  
}  
else if (<expression booléenne 4>)  
{  
    <liste d'instructions 4>  
}
```

Les alternatives imbriquées : Exemple

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int temp;
    printf("Entrez la température de l'eau :");
    scanf("%d",&temp);
    if (temp <= 0)
    {
        printf("C'est de la glace");
    }
    else if (temp < 100)
    {
        printf("C'est du liquide");
    }
    else
    {
        printf("C'est de la vapeur");
    }
    return EXIT.SUCCESS;
}
```

Les alternatives : choix multiple

```
switch (<variable>)  
{  
    case <constante 1> : <liste d'instructions 1>  
                        break;  
    case <constante 2> : <liste d'instructions 2>  
                        break;  
    ...  
    case <constante n> : <liste d'instructions n>  
                        break;  
    default: <liste d'instructions n+1>  
            // pas d'instruction break  
}
```

- Attention en C la variable ne peut être que de type discret (int, char, ...)

Les alternatives : choix multiple

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int a;
    printf("Entrez votre âge :");
    scanf("%d",&a);
    switch(a)
    {
        case 0: printf("Vous êtes un nourrisson.");
                break;
        case 1: printf("Vous avez 1 an.");
                break;
        case 2: printf("Vous avez 2 ans.");
                break;
        default: printf("Vous avez plus de 2 ans.");
    }
    return EXIT_SUCCESS;
}
```